

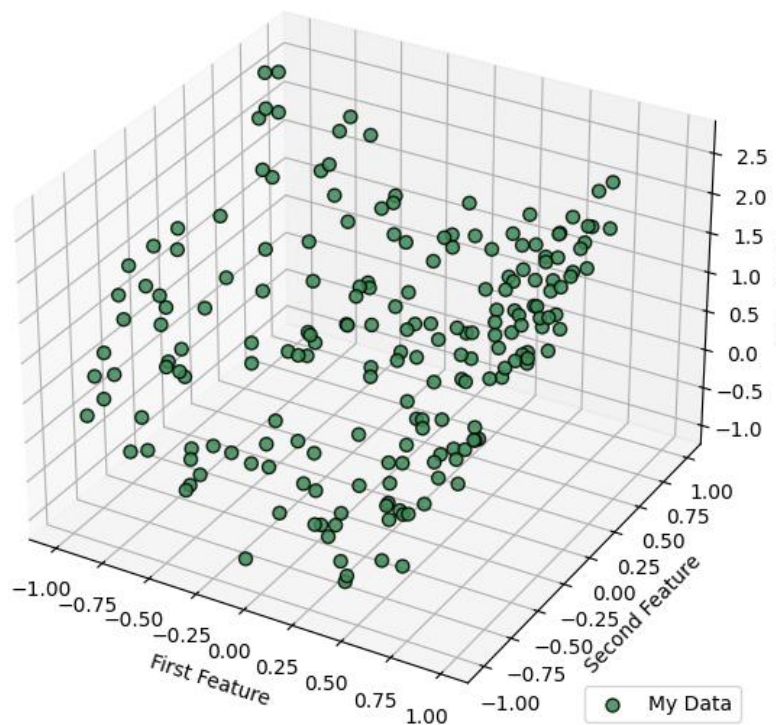
Machine Learning Week 3 Assignmented
By Gaurav Naudiyal (Student ID: 25339213)
Msc Computer Science – Augmented & Virtual Reality

My data id is : # id:14-28-14

Question I (a) Plotting Data (Flat or Curve?)

	first	second	final_score
0	0.99	-0.99	0.894918
1	0.35	-0.89	-0.696835
2	0.47	-0.19	0.276923
3	0.44	-0.68	-0.901706
4	0.67	-0.90	-0.493966

3D Scatter — Week 3 Data



After observing the plot, It seems like the training data lies on a curve, because all the dots do not perfectly fall on a flat surface. Especially around the middle there is a lot of twisting & arching which makes it look like a curve.

(b) Reporting the parameters of the trained models

```
C = 0.1:
Feature      | Coefficient
-----|-----

C = 1:
Feature      | Coefficient
-----|-----

C = 10:
Feature      | Coefficient
-----|-----
second       | 0.558
first^2      | 0.698

C = 100:
Feature      | Coefficient
-----|-----
second       | 0.935
first^2      | 1.840
```

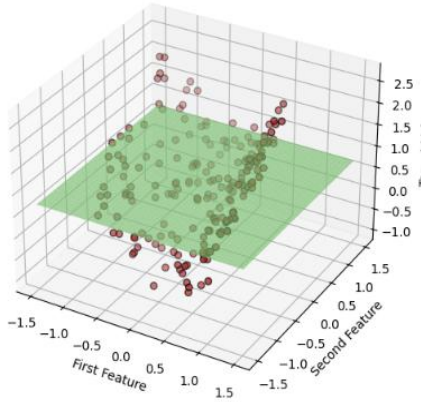
Through the table, **we can see that for very small values of C (C=0.01, C=0.1), almost all of the coefficient values are zero.** This means that the model doesn't rely on any features to make predictions.

However, as soon as the value of C becomes higher (in this case, 10 and 100) we can see that a dramatic shift happens. **A few features escape the penalty and show up with non zero coefficients** (second & first^2).

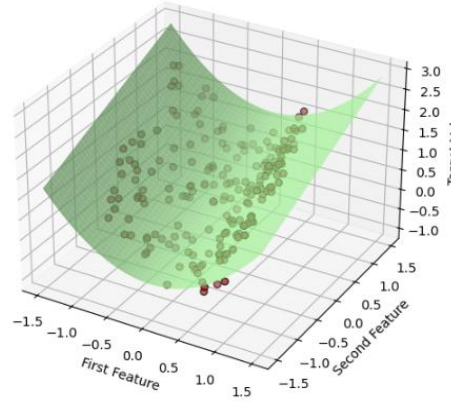
The fact that first^2 survives and not just first suggests a non linear pattern is present while the second is useful as a linear feature

(c) Generating predictions for the target variable

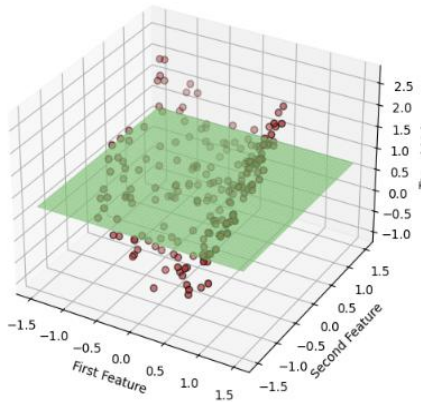
Lasso Prediction Surface (C=0.1)



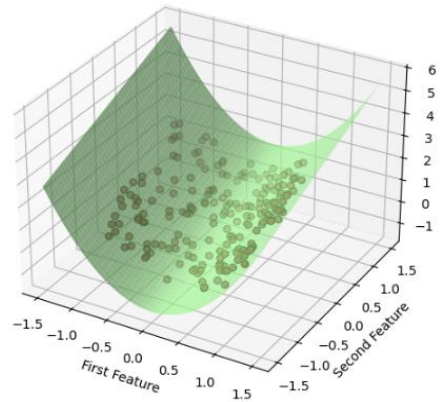
Lasso Prediction Surface (C=10)



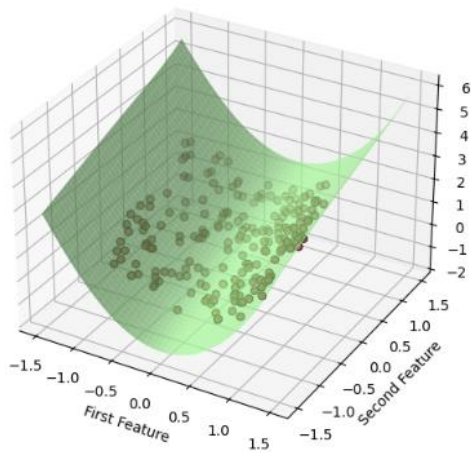
Lasso Prediction Surface (C=1)



Lasso Prediction Surface (C=100)



Lasso Prediction Surface (C=1000)



Through the above five graphs we can notice the following differences:

When $C=0.1$ and 1 , all the parameters of the lasso regression model are equal to 0 , so the prediction points are plotted and the predicted score is negative. But when $C = 10$ and 100 and 1000 some of the parameters of the lasso regression model are not 0 , so the prediction points become curves instead of a flat straight line and the predicted scores are integers. In other words, **As C goes higher, the surface starts to bend and tracking the data becomes better.**

(d) Overfitting & Underfitting

Over-fitting is the term that occurs when the training error of the training set and the training error of the test set is large. In other words, it basically means that the model performs well on the training set and poorly on the test set as the model fit the “noise” in the training data. Under-fitting, on the other hand, occurs when the model performs poorly on both the training and the test set because the model is too simple to be able to capture the behaviour of the data.

Through observations in my answer from (b) and (c), When $C=0.1$ and $C=1$, the parameters of the model are 0 which corresponds to the model having no ability to predict the data and is therefore underfitting. When $C = 100$ the model predicts a higher score than when $C=10$ so the model is still underfitting. However when $C=1000$, the model predicts a lower score than $C=100$ so it means that the model is over-fitting.

(e) Ridge Regression Model

With the ridge regression model, many coefficients are small but rarely zero while on the other hand for Lasso, most coefficients are exactly zero except a few.

In ridge regression model, we can see that as the value of C increases, the number and size of non zero coefficients both increase but even for smaller value of C some coefficients are never zero. This means that ridge regression model spreads the weights more evenly.

```

C = 0.1 (Ridge):
Feature      | Coefficient
-----|-----
first        | 0.029
second       | 0.613
first^2      | 0.812
second^2     | -0.053
first^3      | 0.032
first^2 second | 0.087
first second^2 | -0.049
second^3     | 0.228
first^4      | 0.656
first^2 second^2 | 0.224
first second^3 | -0.012
second^4     | -0.017
first^5      | 0.021
first^2 second^3 | 0.017
first second^4 | -0.041
second^5     | 0.101

```

```

C = 1 (Ridge):
Feature      | Coefficient
-----|-----
second       | 0.876
first^2      | 1.315
first second | -0.087
second^2     | -0.090
first second^2 | -0.014
second^3     | 0.129
first^4      | 0.575
first^3 second | 0.106
first^2 second^2 | 0.234
first second^3 | 0.043
first^5      | 0.051
first^4 second | 0.017
first^3 second^2 | 0.171
first second^4 | -0.110
second^5     | 0.015

```

```

C = 10 (Ridge):
Feature      | Coefficient
-----|-----
first        | 0.074
second       | 1.043
first^2      | 1.758
first second | -0.237
second^2     | -0.122
first^3      | -0.270
first^2 second | -0.355
first second^2 | -0.090
second^3     | -0.116
first^4      | 0.211
first^3 second | 0.224
first^2 second^2 | 0.163
first second^3 | 0.178
second^4     | 0.070
first^5      | 0.196
first^4 second | 0.257
first^3 second^2 | 0.538
first^2 second^3 | 0.199
first second^4 | -0.203
second^5     | 0.112

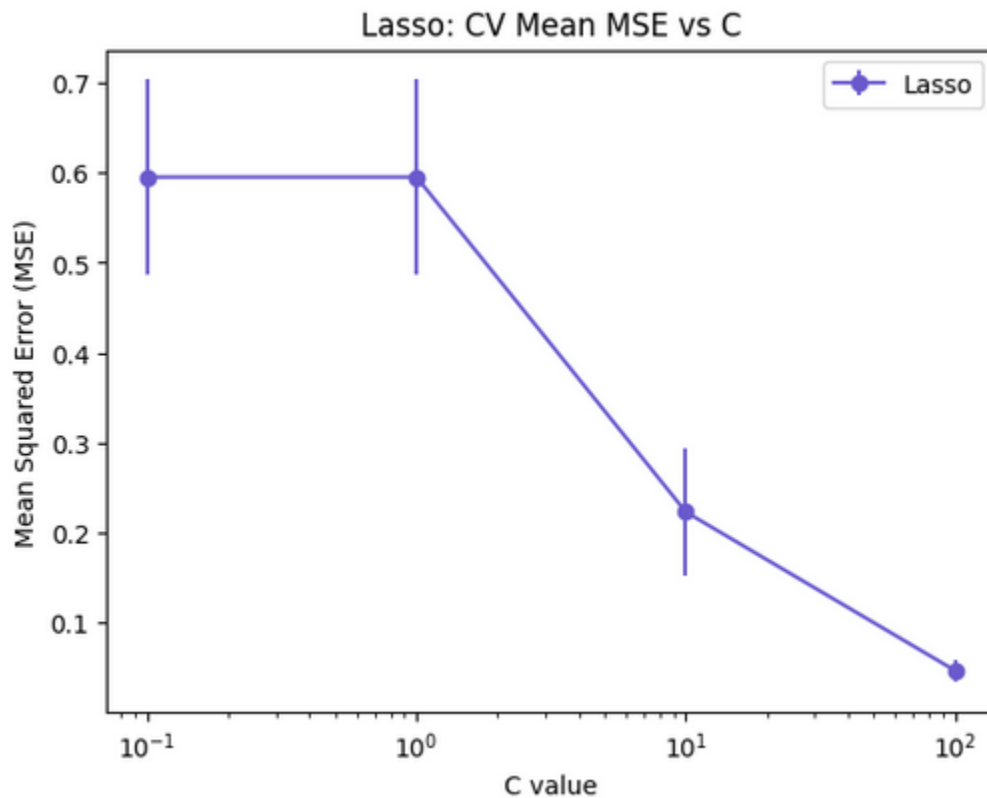
```

```

C = 100 (Ridge):
Feature      | Coefficient
-----|-----
first        | 0.166
second       | 1.143
first^2      | 1.832
first second | -0.315
second^2     | -0.160
first^3      | -0.579
first^2 second | -0.677
first second^2 | -0.239
second^3     | -0.323
first^4      | 0.151
first^3 second | 0.308
first^2 second^2 | 0.204
first second^3 | 0.246
second^4     | 0.108
first^5      | 0.411
first^4 second | 0.523
first^3 second^2 | 0.791
first^2 second^3 | 0.380
first second^4 | -0.170
second^5     | 0.239

```

Question ii (a) 5-fold Cross validation

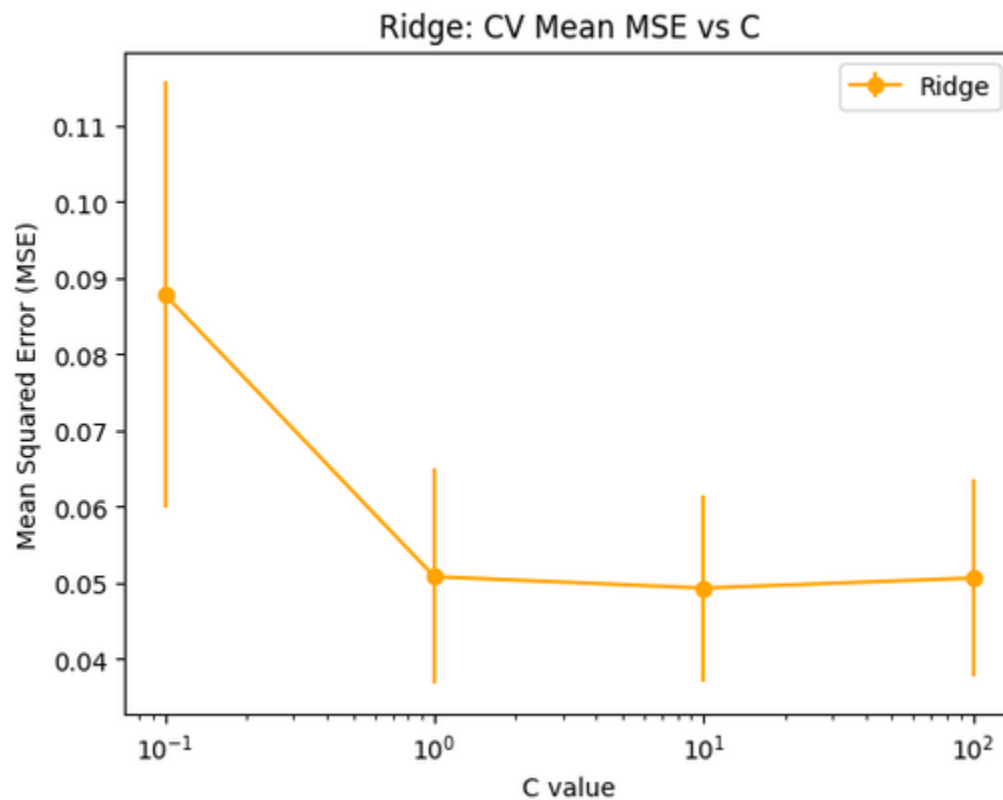


After running 5-fold cross validation, I would choose range of C as [0.1,1,10,100] because it covers both strong and weak regularized models. The best performance of C is between 10 to 100 and gives consistent results across folds.

(b)

Based on the cross validation data, I would recommend using $C=50$ because in the graph we can see that as C gets smaller the mean error gets larger which means that the instability of the predictions also get higher. When C is greater than 50, we can see from the graph that the average error is hardly reduced. The larger the C the higher the probability of overfitting, so it is better to choose a smaller C when the prediction scores of $C=50$ and $C=100$ are close.

(c)



The code idea is exactly the same but the ML model is changed to ridge regression model. I recommend using $C=1$ because the error of $C=1$ is relatively small and the prediction score is high.

Appendix:

(i)

(a)

```
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

df = pd.read_csv('week3.csv', header=None, skiprows=1)
```

```

df.columns = ['first', 'second', 'final_score']

# Just to be sure, always peek at the data
print(df.head())

fig = plt.figure(figsize=(9, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(df['first'], df['second'], df['final_score'],
           c='seagreen',
           s=42,          # Slightly larger dots for style
           edgecolor='black',
           alpha=0.85,    # Not fully opaque, so some overlap is visible
           label='My Data')

ax.set_xlabel('First Feature')
ax.set_ylabel('Second Feature')
ax.set_zlabel('Target Value')
ax.set_title('3D Scatter - Week 3 Data')
ax.legend(loc='lower right')
plt.show()

```

(c)

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso

# Load and label data
df = pd.read_csv('week3.csv', header=None, skiprows=1)
df.columns = ['first', 'second', 'final_score']
X = df[['first', 'second']].values

```

```

y = df['final_score'].values

# Polynomial features, up to 5th degree
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)

# Model parameters
C_list = [0.1, 1, 10, 100]
feature_names = poly.get_feature_names_out(['first', 'second'])

for C in C_list:
    lasso = Lasso(alpha=1/C, max_iter=10000)
    lasso.fit(X_poly, y)
    coefs = lasso.coef_
    print(f"\nC = {C}:")
    print("Feature          | Coefficient")
    print("-----|-----")

    # Print only the ones that are above a small threshold
    for fname, coeff in zip(feature_names, coefs):
        if abs(coeff) > 0.01:          # Only display 'important' features
            print(f"{fname:14} | {coeff:.3f}")

import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

# Setting up a grid (from -1.5 to 1.5) is usually safe for your dataset range
grid = np.linspace(-1.5, 1.5, 50)
f1, f2 = np.meshgrid(grid, grid)
X_grid = np.column_stack([f1.ravel(), f2.ravel()])
X_grid_poly = poly.transform(X_grid)

```

```

for C in [0.1, 1, 10, 100, 1000]:
    lasso = Lasso(alpha=1/C, max_iter=10000)
    lasso.fit(X_poly, y)
    preds = lasso.predict(X_grid_poly).reshape(f1.shape)

    fig = plt.figure(figsize=(8,6))
    ax = fig.add_subplot(111, projection='3d')

    # Surface in 'palegreen', points in 'firebrick'
    ax.plot_surface(f1, f2, preds, color='palegreen', alpha=0.7)
    ax.scatter(df['first'], df['second'], df['final_score'],
               c='firebrick', s=28, edgecolor='black')

    ax.set_xlabel('First Feature')
    ax.set_ylabel('Second Feature')
    ax.set_zlabel('Target Value')
    ax.set_title(f'Lasso Prediction Surface (C={C})')
    plt.show()

```

(b)

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

# Load and label your data (reuse if already loaded)
df = pd.read_csv('week3.csv', header=None, skiprows=1)
df.columns = ['first', 'second', 'final_score']
X = df[['first', 'second']].values
y = df['final_score'].values

# Polynomial features (degree 5)

```

```

poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(['first', 'second'])

ridge_C_list = [0.1, 1, 10, 100]

for C in ridge_C_list:
    # Ridge uses alpha directly as penalty, the inverse of C
    ridge = Ridge(alpha=1/C, max_iter=10000)
    ridge.fit(X_poly, y)
    coefs = ridge.coef_
    print(f"\nC = {C} (Ridge):")
    print("Feature          | Coefficient")
    print("-----|-----")
    for fname, coeff in zip(feature_names, coefs):
        if abs(coeff) > 0.01:
            print(f"{fname:14} | {coeff:.3f}")

```

(ii)

(a)

```

import pandas as pd
import numpy as np

from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

# Data loading
df = pd.read_csv('week3.csv', header=None, skiprows=1)
df.columns = ['first', 'second', 'final_score']
X = df[['first', 'second']].values

```

```

y = df['final_score'].values

# Polynomial features
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)

C_list = [0.1, 1, 10, 100]
lasso_means = []
lasso_stds = []

for C in C_list:
    lasso = Lasso(alpha=1/C, max_iter=10000)
    # Negative mean squared error, flipped
    scores = cross_val_score(lasso, X_poly, y, cv=5, scoring='neg_mean_squared_error')
    lasso_means.append(-np.mean(scores))
    lasso_stds.append(np.std(scores))

plt.errorbar(C_list, lasso_means, yerr=lasso_stds, fmt='-o', color='slateblue',
label='Lasso')

plt.xlabel('C value')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('Lasso: CV Mean MSE vs C')
plt.xscale('log')
plt.legend()
plt.show()

```

(c)

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

```

```

from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt

# Data loading
df = pd.read_csv('week3.csv', header=None, skiprows=1)
df.columns = ['first', 'second', 'final_score']
X = df[['first', 'second']].values
y = df['final_score'].values

# Polynomial features
poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)

C_list = [0.1, 1, 10, 100]
ridge_means = []
ridge_stds = []

for C in C_list:
    ridge = Ridge(alpha=1/C, max_iter=10000)
    scores = cross_val_score(ridge, X_poly, y, cv=5, scoring='neg_mean_squared_error')
    ridge_means.append(-np.mean(scores))
    ridge_stds.append(np.std(scores))

plt.errorbar(C_list, ridge_means, yerr=ridge_stds, fmt='-o', color='orange',
label='Ridge')

plt.xlabel('C value')
plt.ylabel('Mean Squared Error (MSE)')
plt.title('Ridge: CV Mean MSE vs C')
plt.xscale('log')
plt.legend()
plt.show()

```